

«Навигатор по смыслу» — сравнение моделей семантического поиска

Контекст задачи

Проблема: Обычный поиск по коду ищет точное совпадение текста. Написал «обработка ошибок» — а в коде `exception_handler`, и поиск ничего не нашёл.

Решение: Семантический поиск на основе эмбедингов (числовые векторы, кодирующие смысл). Похожие фрагменты оказываются близко в математическом пространстве.

Задача команды: Сравнить две `embedding`-модели на маленьком датасете и ответить — какая работает лучше и почему.

Подсказка: Весь код пишется в Jupyter Notebook. Никаких сервисов, фронтенда и Docker — только ноутбук и выводы.

Формат сдачи: Архив с кодом и пояснительной запиской о проекте

Задание

Шаг 1. Загрузка данных и моделей

Данные: Датасет из 100 фрагментов кода на Python с коротким описанием каждой функции.

Действие: загрузить датасет и выбрать две `embedding`-модели (либо одну из предложенных + одну свою с обоснованием).

Технически:

```
pip install sentence-transformers
```

```
from sentence_transformers import SentenceTransformer
```

```
model = SentenceTransformer("имя-модели")
```

Исследование состоит из трёх шагов. Каждый помещается в один раздел Jupyter Notebook.

Шаг 2. Генерация эмбедингов и поиск

Действие для каждой модели:

1. Превратить все 100 фрагментов кода в векторы (эмбединги) и сохранить их
2. Взять тестовый набор из 15 вопросов с известными правильными ответами
3. Для каждого вопроса найти топ-3 самых похожих фрагмента по косинусному сходству

Косинусное сходство:

```
# numpy вариант
```

```
np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

```
# sentence-transformers (пакетно)

from sentence_transformers import util

util.cos_sim(a, b)
```

Шаг 3. Метрика и визуализация

Метрика Precision@3 — доля вопросов, для которых правильный ответ оказался в топ-3 выдачи.

Результат: Сводная таблица «Модель / Precision@3 ».

Визуализация: Двумерная проекция эмбедингов лучшей модели с помощью t-SNE. Каждая точка раскрашена по тематике (авторизация, БД, HTTP и т.д.).

```
from sklearn.manifold import TSNE

coords = TSNE(n_components=2).fit_transform(embeddings)

# график через matplotlib
```

Предоставляемые данные

Файл	Содержимое	Формат
code_corpus.json	100 фрагментов Python-кода: id, function_name, code, description, category	JSON
eval_questions.json	15 вопросов с полем correct_chunk_id	JSON
starter.ipynb	Стартовый ноутбук со структурой шагов и TODO-блоками	Jupyter

Требования к сдаче

Обязательные

- Ноутбук запускается без ошибок на Python 3.12 после `pip install -r requirements.txt`.
- Сравнены две embedding-модели, для каждой вычислен Precision@3 .
- Построена таблица сравнения моделей и график t-SNE с раскраской по категориям.
- В финальной ячейке ноутбука — вывод в 3-5 предложениях: какая модель выбрана и почему.

Дополнительные

- Сравнение трёх моделей вместо двух с обоснованием выбора третьей.
- Анализ ошибок: на каких типах вопросов модель путается чаще всего.
- Сравнение качества поиска при вопросах на русском и английском языках.

Рекомендованный стек и формат сдачи

Компонент	Технология	Примечание
Язык	Python 3.12	Ничего кроме Python
Эмбединги	sentence-transformers	Обязательная библиотека
Вычисления	numpy, pandas	Для метрик и таблиц
Визуализация	matplotlib + sklearn TSNE	Интерактивность (plotly) — по желанию
Формат	Jupyter Notebook	Основной артефакт сдачи

Подсказка: Рекомендуемые модели для старта: paraphrase-multilingual-MiniLM-L12-v2 (быстрая, понимает русский) и paraphrase-multilingual-mpnet-base-v2 (медленнее, но качественнее). Обе ставятся через sentence-transformers одной строкой.

Программа демонстрации: в архиве должны присутствовать таблица сравнения моделей, график кластеров и текстовый файл с финальным выводом. Эксперты почти наверняка спросят «почему эта модель лучше?» — готовый ответ в 1-2 предложения стоит заготовить заранее.